

Key Decisions

For Cognition. What I built, why that scope, and where the reasoning changed. Repo: github.com/rythama/internal-tools-platform

The reframe that set the scope

"Can Devin build these tools" is not the question. CRUD over Postgres is not hard. The real question is whether owning ~13 internal tools costs less than \$250K/year after you count the platform under them, the engineer who owns it, and the compliance surface it pulls into scope.

So I scoped around the eleventh tool, not the first: one console hosting many tools, each a spec file over four shared primitives. Deny-by-default authorization. Hash-chained audit. PII masking with audited break-glass. Maker-checker approvals. I anchored on the KYC queue because it forces all four primitives to carry real load at once.

I deliberately did not replicate the two things Power Apps is genuinely good at: citizen development and the connector catalog. Devin makes engineer-hours cheap. It does not turn an ops analyst into an app author.

Three architectural decisions

Guardrails before agents. I wrote the architecture doc, schema, core contract, and CI by hand before the first session. An agent's output quality is bounded by what the repo can prove about it.

Interface-first, so sessions run concurrently. I declared the core contract up front and ran two sessions in parallel against it: one implementing the primitives, one building the console. When they merged, the console bound to the real implementation with zero console edits. That is the throughput argument, demonstrated instead of asserted.

Masking as a structural property. The read API masks internally and takes no unmask parameter. An API that lets the caller opt out of masking is an API that leaks PII.

The thesis changed twice

I started with "build a framework." The numbers killed it. Marginal per-tool cost is ~\$40K custom vs ~\$39K built properly in Power Apps. Parity. Only ~12% of per-tool work is agent-compressible; requirements, security review, UAT, and cutover are not. The premise my architecture was organized around was wrong, and ARCHITECTURE.md now says so.

I moved to "own the primitives, rent the UI." A red-team pass then found six load-bearing errors in my own model, including a 3.2x maintenance penalty on the incumbent I never justified, and a breakeven computed against a baseline my own first recommendation would have corrected. Fixed, the build never breaks even at any tool count, discount rate, or horizon.

So the recommendation is not a saving. It is a capability purchase at ~\$400K/yr: renting the UI alone buys ~65% of the primitives at ~\$125–145K/yr, and owning the primitives buys the last ~30% for ~\$260–275K/yr. Sell it as cost-cutting and it dies in the room.

Time, honestly accounted

The prototype fits the 2-hour bound: ~30 minutes of human seeding plus ~66 minutes of agent time across six sessions (~7,000 lines, 228 tests, \$27 measured agent cost). The commit history spans longer because analysis and presentation polish continued afterward. The UI design pass was human-and-assistant work, not Devin's, and DEVIN-WORKFLOW.md says so. Better stated than reconstructed from timestamps.

What the prototype measured

Two findings, both against me.

The verification harness proved none of its advertised invariants. Mutation testing: delete PII masking, all 54 tests pass. Defeat the two-person rule, pass. Remove the four-eyes gate, pass. The lint rule I called machine-checked is evadable five ways, including the idiom the repo's own policy module uses.

Review does not scale with lines. It scales with load-bearing claims, and reading does not find them. Every confirmed defect was correct-looking code wrong in one conjunct. Finding them required executing mutations, not reading diffs. PR #1 merged unread, 17 seconds after opening, because CI was green. The review tax was not paid. It was skipped, invisibly.

The fix worked: tests that assert audit rows and state transitions now kill 5/5 mutants in CI. That only happened because we measured.

What I'd flag

I over-ranked one finding: the hardcoded session secret is major, not critical, because the dev role-switcher hands out any role by design. The genuine critical was a two-person rule counting votes instead of voters.

The prototype demonstrates the primitives, not the recommendation. The crux, binding a rented UI's queries to an identity it cannot forge, is proven here only at the API layer. The database half is the pilot's week-two gate. That plus mutation testing is where the next two hours go. Not more tools.